POC de triage médical bilingue

Rapport technique et recommandations

OCR Projet 14 · 10 novembre 2026 · résultats acquis, variantes et décision de livraison

Je construis un prototype de triage français / anglais basé sur Qwen3-1.7B-Base. Ma démarche relie la préparation des données, un SFT successif puis mixte, un pilote DPO généraliste et une démonstration traçable. Je teste ensuite un DPO ciblé sur les erreurs du train. Je distingue les résultats techniques, l'accord avec des annotations de modèles et une validation clinique, qui n'a pas été réalisée.

Décision actuelle

Je retiens et je déploie le SFT mixte à l'époque 3 : il préserve bien mieux la QA que le SFT successif, avec une macro-F1 de triage légèrement supérieure et un rappel urgent inférieur sur validation (91 contre 95 sur 100). Depuis le 10 novembre, le Space et la configuration de l'API servent cet adaptateur. La poursuite jusqu'à dix époques n'apporte pas de meilleur compromis global. Le DPO ciblé est exécuté : le rappel urgent recule de 91 % à 87 %, puis 83 %. Je ne le retiens pas.

Résultats et preuves disponibles

Élément	Constat
Corpus SFT nettoyé	4 974 entrées, dont 3 986 pour l'entraînement
Triage de validation	Macro-F1 priorité 0,7715 ; rappel urgence maximale 95/100
DPO généraliste	256/64 paires ; 32 pas ; pas de gain de triage observé
SFT mixte, époque 3	Validation : macro-F1 0,7874 ; urgents 91/100 ; QCM 57,25 %
DPO ciblé	355/85 paires ; 32 pas ; rappel urgent et macro-F1 dégradés
Démonstration	Space Hugging Face ZeroGPU sur le SFT mixte époque 3 ; inférence réelle vérifiée après bascule
CI/CD	Tests, image Docker et déploiement Space réussis sur GitHub Actions
Test final	976 prédictions conservées ; SFT successif : macro-F1 priorité 0,7858, urgents 88/98
API vLLM publique	10 novembre, Colab L4 + HTTPS, SFT mixte : médiane HTTP 1,23 s ; 8 requêtes simultanées en 1,54 s, soit 329 tokens/s

Le test final et la vérification vLLM sont exécutés et leurs preuves récupérées ; ils portent sur le SFT successif, sélectionné avant l'expérience mixte. Le test ayant déjà été consulté, je ne le rejoue pas pour le mixte : sa sélection repose sur la validation. L'endpoint vLLM public est exécuté et vérifié le 10 novembre avec le SFT mixte. Trois limites de livraison subsistent : la démonstration permanente utilise Transformers, l'endpoint vLLM est temporaire (GPU Colab loué à la demande, URL différente à chaque lancement), et les poids finaux du pilote DPO généraliste sont absents de la sauvegarde Drive accessible. Je les documente sans revendiquer une conformité intégrale.

Trajet de lecture

Choix techniques → données et consensus → séparation des tâches → correction du protocole → comparaison SFT/DPO → risques → architecture de démonstration → coûts → preuves et recommandations.

Les notebooks 01 à 09 suivent la chronologie, avec un repère de lecture à chaque entrée. Les corpus et les poids restent hors de Git ; les scripts, contrats, protocoles et notebooks sont versionnés dans le dépôt GitHub, public depuis le 10 novembre.

Choix techniques et justification

Choix	Raison	Limite acceptée
Qwen3-1.7B-Base	Socle imposé par la phase 1 ; multilingue, Apache 2.0, assez petit pour itérer en quelques minutes sur un GPU de location	Connaissances médicales limitées ; le socle seul ne produit pas le format attendu
LoRA (PEFT), rang 16	17,4 M de paramètres entraînés, environ 1 % ; adaptateur d'environ 70 Mo, versionnable et interchangeable sur un même socle	Capacité d'adaptation bornée ; oubli observé en entraînement successif
SFT puis DPO (TRL)	Le SFT apprend le format et la tâche ; le DPO ajuste une préférence entre deux réponses sans modèle de récompense	Le DPO dépend entièrement de la qualité et de l'équilibre des paires
vLLM	Traitement des requêtes par lots continus, API compatible OpenAI, chargement d'adaptateurs LoRA sur un socle partagé : meilleur débit qu'une génération Transformers requête par requête	Demande un GPU réservé ; incompatible avec l'allocation à la demande de ZeroGPU
FastAPI, SQLite, Docker	Contrat typé, journal simple à auditer, image reproductible	SQLite en clair, mono-instance : suffisant pour un POC seulement

1. Construire un corpus adapté à la tâche

Je commence par explorer chaque source. Une présence de contenu médical ne suffit pas à établir son utilité pour le triage ; je recherche une présentation disponible à l'accueil et une cible exploitable.

Source	Observation	Usage retenu
MediQAl, FR	QCM corrigés et cas ; CC BY 4.0 constatée	QA et candidats au triage
FrenchMedMCQA, FR	QCM ; licence non renseignée	Exploration, sans ajout au corpus final
MedQuAD, EN	Questions et réponses rédigées ; licence non renseignée	QA expérimentale, droits à clarifier
UltraMedical-Preference, EN	Préférences natives ; MIT constatée	DPO et candidats au triage

Aucune de ces sources ne fournit directement les étiquettes ESI nécessaires au projet. Je crée donc un corpus dérivé, avec une qualification temporelle et des annotations indépendantes de Codex, Claude et Mistral. Les schémas contrôlent les citations, les énumérations, les abstentions et la cohérence urgence / ESI.

Je retiens une majorité sur la qualification, la priorité puis l'ESI exact. Mistral intervient comme troisième vote ; pour UltraMedical, les appels sont ciblés sur les désaccords. Une majorité sur la priorité seule ne suffit pas pour une réponse cible qui contient un ESI précis.

Origine	Cas candidats	Cas uniques retenus	Entrées après traduction
MediQAl	2 007	700	1 400
UltraMedical	2 000	489	978

Les traductions permettent de présenter le même raisonnement dans les deux langues. Elles ne doublent pas le nombre de situations indépendantes. Le volume résulte de la qualité des candidats ; je ne relâche pas le consensus pour atteindre un quota.

Protection et provenance

Ma démarche RGPD suit quatre principes. **Origine** : je n'utilise que des corpus publics de vignettes pédagogiques et de questions d'examen, sans dossier patient réel. **Minimisation** : je ne conserve que le texte, la langue, la source, la licence et les étiquettes ; aucune donnée n'est collectée auprès de personnes. **Contrôle** : auditer_anonymisation.py applique Presidio 2.2 avec spaCy fr_core_news_md et en_core_web_sm aux 10 000 messages des 5 000 exemples initiaux, seuil 0,65. Il relève 16 386 détections, surtout des lieux (9 655), des dates (3 541) et des « personnes » (3 183). Sur un échantillon de 60 de ces dernières, je trouve surtout des éponymes médicaux (Babinski, Lasègue), des termes cliniques et des médicaments, plus quelques désignations de vignettes pédagogiques (« Monsieur X. », « Marianne G. ») ; aucun téléphone, courriel, adresse IP, numéro RPPS ou licence médicale. Je ne masque donc pas automatiquement : remplacer « maladie de Parkinson » par une balise dégraderait le corpus sans protéger personne. **Usage** : la démonstration et les journaux n'acceptent que des cas fictifs.

Une revue humaine reste nécessaire : cet audit ne constitue pas une preuve d'anonymisation complète ni de conformité RGPD. Un usage réel imposerait une analyse d'impact, une base légale, un hébergement certifié HDS et un journal chiffré. Les licences non renseignées doivent être clarifiées avant une diffusion des données dérivées.

Les annotations constituent un standard silver. L'indépendance des appels ne garantit pas l'indépendance des biais. Les cas exclus peuvent aussi rendre le jeu retenu plus facile que les situations rencontrées à l'accueil.

2. Deux apprentissages et des partitions figées

Je sépare l'apprentissage de réponses médicales générales et celui du triage. Les exemples partagent le schéma messages system / user / assistant, mais chaque phase conserve son objectif et ses partitions.

La préparation initiale produit 2 622 QA et 2 378 triages, soit 5 000 entrées et 50 % de chaque langue. L'audit ultérieur retire les prompts communs entre groupes et les cinq paires de test déjà exposées dans un pilote. Je conserve les versions historiques et je fige les partitions v3.

Corpus v3	Train	Validation	Test	Total
QA	2 098	262	262	2 622
Triage	1 888	238	226	2 352
Total SFT	3 986	500	488	4 974

Le total comprend les jeux d'évaluation : il ne désigne pas 4 974 exemples utilisés pour calculer les gradients. L'écart au volume initial est justifié par l'élimination des fuites. Les versions FR et EN d'un même cas restent dans le même split ; une exclusion s'applique à toute la paire.

Corpus DPO

Je conserve les préférences chosen/rejected natives d'UltraMedical, en excluant les sollicitations SFT et les chevauchements détectés. La version nettoyée compte 4 470 paires de train et 494 de validation. Le pilote en utilise 256 et 64, stratifiées par catégorie de préférence. Une paire du train dépasse la longueur autorisée et sort de l'éligibilité, sans troncature de la réponse.

Reproductibilité

Je conserve identifiants, groupes bilingues, provenance, empreintes SHA256, seed et manifeste d'exclusions. La version v3 ne réécrit pas les partitions historiques. Le script figer_partitions.py et ses tests vérifient les frontières. La comparaison finale doit utiliser les empreintes inscrites dans protocole_test_final.json.

Le volume équilibré entre langues est un choix de constitution. Il ne suffit pas à garantir une qualité équivalente en français et en anglais ; cette hypothèse est évaluée séparément.

3. Apprendre, diagnostiquer, corriger

Protocole SFT

J'adapte Qwen3-1.7B-Base, révision ea980cb0a6c2ae4b936e82123acc929f1cec04c1, avec LoRA. Le socle reste figé. Je supervise uniquement les tokens de la réponse assistant et je partage la construction du prompt avec l'inférence.

Les premiers essais montrent une incohérence entre le format d'entraînement et l'arrêt attendu. Je corrige le protocole avec le même préfixe et l'EOS natif du socle, identifiant 151643. Les exemples trop longs sont refusés plutôt que tronqués. Cette correction est vérifiée avant la poursuite des entraînements ; les sorties initiales restent conservées.

Le parcours corrigé est : socle → LoRA QA → poursuite du même LoRA sur le triage → pilote DPO. Le SFT triage v3 repart de la phase QA corrigée, après nettoyage des partitions.

Paramètre SFT triage v3	Valeur
Nombre de pas / batch effectif	357 / 16
Learning rate / warmup	0,0001 / 20 pas
Précision / seed	BF16 / 3407
Longueur maximale	1 024 tokens
Paramètres entraînables	17 432 576, environ 1 % du modèle chargé
Durée observée Colab	222 s, entraînement et validation finale

Je mesure aussi un rappel de QA pendant la spécialisation : 10 % des exemples du mélange, représentant environ 28 % des tokens de réponse. Cet essai met en évidence un compromis ; je n'en fais pas un remplacement automatique du triage seul.

Pilote DPO

Le DPO compare les réponses préférées et rejetées à une référence SFT figée. Désactiver le LoRA de référence utiliserait le socle et changerait l'expérience. Un test CPU miniature vérifie la mise à jour de la politique, les frontières de tokens et l'immobilité de la référence.

Le pilote utilise 32 pas, un batch effectif de 16, un learning rate de 0,000005 et beta = 0,1. Le checkpoint 32 est retenu par la loss de validation. La durée totale observée est de 100 s, dont 69 s d'optimisation. Ces durées sont propres à la session G4 ; elles ne constituent pas un benchmark matériel général.

4. Mesurer la tâche et ses effets secondaires

La macro-F1 des trois priorités constitue la mesure principale : elle traite chaque classe à poids égal. Je complète cette moyenne par le rappel des urgents, le sous-triage critique, l'accuracy ESI, les sorties analysables et la cohérence bilingue. La QA reste évaluée pour détecter l'oubli après spécialisation.

Le sous-triage critique désigne ici un ESI de référence 1 ou 2 prédit 3 à 5. Je distingue la proportion parmi les sorties analysables et celle parmi les urgents de référence. Une sortie non analysable reste une erreur de classification.

Validation comparée : 238 triages et 262 QA

Mesure	SFT v3	DPO pilote
Macro-F1 priorité	0,7715	0,7674
Accuracy ESI	0,7311	0,7269
Rappel urgence maximale	95/100	95/100
Sous-triages critiques	5, sur 3 cas distincts	Les mêmes 5
Accord bilingue ESI	88/119	91/119
QCM QA corrects	5/131	5/131
F1 lexical QA ouverte	0,0359	0,0363

La loss DPO de validation passe de 0,6931 à 0,5361 ; les préférences sont correctement ordonnées pour 51/64 paires. Cela ne se traduit pas par une amélioration du triage. Je conserve le SFT v3, sans interpréter le faible écart de macro-F1 comme une différence statistiquement établie.

La QA corrigée obtenait auparavant 59,54 % aux QCM et 0,3762 de F1 lexical. Sa dégradation après le triage empêche de présenter le modèle final comme un assistant médical généraliste. La relecture montre aussi des justifications introduisant une information absente, par exemple une fièvre : un format correct ne suffit pas.

Le SFT v3 obtient 0,8180 de macro-F1 en français et 0,7229 en anglais. Pour quantifier l'incertitude dans une étude ultérieure, je rééchantillonnerais les cas par paire, car les deux langues ne sont pas indépendantes.

5. Test final du SFT successif

Le protocole est figé avant les calculs : 226 triages et 262 QA, décodage déterministe BF16, batch 32, EOS natif et limites de 192/256 tokens. Le candidat reste celui choisi sur validation. Je récupère les 976 prédictions (488 par modèle), vérifie leur ordre et recalcule les métriques localement : les résultats sont identiques aux rapports Colab. Le DPO n'est pas évalué sur ce test, car ses poids finaux sont absents de Drive.

Test figé	Socle	SFT triage v3
Macro-F1 priorité	0,0556	0,7858
Accuracy ESI	3,10 %	74,78 %
ESI analysables	10,62 %	100 %
Urgents retrouvés	0/98	88/98
Accord ESI FR/EN	Aucune paire doublement analysable	89/113
QCM corrects	1/131	0/131
F1 lexical QA ouverte	0,1376	0,0323

Le gain sur le socle inclut l'apprentissage du format de réponse. Je ne l'interprète pas comme une mesure isolée de connaissances médicales. Le SFT retrouve 88 urgents sur 98 : les 10 sous-triages critiques concernent 7 cas distincts. Ils représentent 10,20 % des urgents de référence, contre 5 % sur validation. La macro-F1 globale légèrement supérieure ne signifie donc pas une meilleure reconnaissance des situations urgentes.

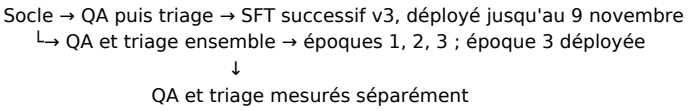
La macro-F1 priorité est de 0,7804 en français et 0,7911 en anglais ; le rappel des urgents est respectivement de 42/49 et 46/49. Le classement relatif des langues change selon la partition. Les deux traductions d'un cas ne sont pas des observations indépendantes.

L'absence de réponse QCM correcte confirme la perte de la capacité QA après la spécialisation. Ce résultat motive l'expérience suivante. Les variantes ultérieures, dont le SFT mixte finalement déployé, sont comparées sur validation uniquement ; puisque le test a déjà été consulté, une confirmation indépendante demanderait de nouveaux cas.

6. Réduire l'oubli avec un seul SFT mixte

Hypothèse et protocole

Je repars du socle avec un nouveau LoRA et l'union des deux trains v3 : 2 098 QA et 1 888 triages. Les 3 986 exemples sont mélangés sans répétition artificielle, avec autant de français que d'anglais. Je conserve la validation v3, le prompt et l'EOS. Cette expérience change la recette complète ; elle n'isole pas causalement le seul ordre des tâches.



Je réalise trois époques, batch effectif 16, taux maximal 0,0001, warmup 20 étapes puis décroissance cosinus. Les checkpoints 250, 500 et 750 correspondent à la fin de chaque époque. Le contexte maximal est 5 120 tokens ; aucun exemple n'est tronqué. Le QA représente 53 % des exemples mais environ 79 % des tokens supervisés : un mélange d'exemples n'est pas un équilibre de gradient.

Le premier calcul sur A100 40 Go dépasse la VRAM au pas 62. Le calcul complet sur G4 est conservé et dure environ 16 min 25 s, évaluations comprises. Cette durée ne constitue pas une comparaison contrôlée des GPU.

Résultats sur la même validation

Modèle	QCM	F1 lexical QA	Macro-F1 priorité	Urgents retrouvés
SFT successif v3	3,82 %	0,0359	0,7715	95/100
Mixte, époque 1	53,44 %	0,3364	0,6680	76/100
Mixte, époque 2	58,02 %	0,3571	0,7720	85/100
Mixte, époque 3	57,25 %	0,4341	0,7874	91/100

Le mélange conserve mieux les deux tâches. L'époque 3 constitue un candidat plus équilibré, mais pas supérieur sur toutes les mesures : quatre entrées urgentes supplémentaires sont manquées par rapport au SFT successif. Les neuf erreurs touchent sept cas, tous classés ESI 3 au lieu de 2. Cinq erreurs existaient déjà ; quatre apparaissent avec le mélange.

Je ne transforme pas ces neuf entrées de validation en données d'entraînement. La QA de validation confond en partie langue et tâche : les 131 QCM sont français et les 131 réponses ouvertes anglaises. Le F1 lexical ne mesure pas à lui seul la justesse médicale.

7. Prolonger jusqu'à dix époques

Je repars des poids exacts de l'époque 3, puis effectue sept passages supplémentaires avec un taux maximal réduit à 0,00002, microbatch 2 et accumulation 8. L'optimiseur et le calendrier sont réinitialisés : ce n'est pas la reprise exacte d'un entraînement planifié dès le départ sur dix époques.

Époque cumulée	QCM	F1 QA	Macro-F1 priorité	Rappel urgent	Sur-triage
3, référence	57,25 %	0,4341	0,7874	91 %	18,07 %
4	54,96 %	0,4366	0,7792	82 %	17,23 %
5	56,49 %	0,4354	0,8050	86 %	14,29 %
6	55,73 %	0,4626	0,7711	92 %	19,75 %
7	54,96 %	0,4686	0,7824	90 %	18,07 %
8	52,67 %	0,4992	0,7689	90 %	19,75 %
9	53,44 %	0,4743	0,7740	89 %	18,49 %
10	53,44 %	0,4798	0,7740	89 %	18,49 %

L'époque 5 améliore la macro-F1 tout en retrouvant moins d'urgents. L'époque 6 retrouve une entrée urgente supplémentaire mais réduit la macro-F1. L'époque 8 améliore la métrique lexicale QA, sans gain de triage. Aucun checkpoint ne domine l'ensemble des critères.

La loss de validation passe d'environ 0,4794 à l'époque 4 à 0,5115 à l'époque 10. Cette évolution est compatible avec du surapprentissage, sans constituer une preuve causale. Les 3 500 prédictions et les 14 rapports sont récupérés et recalculés localement. Le calcul et ses évaluations durent environ 46 min 49 s.

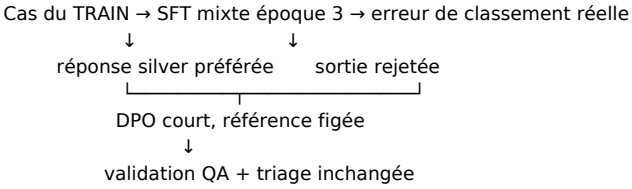
Point de traçabilité : le dossier final adapter/ contient les poids restaurés de l'époque 4, meilleure loss de cette poursuite, et non ceux de l'époque 10. Les comparaisons utilisent les checkpoints explicites. Les paires FR/EN ne sont pas indépendantes ; une variation d'une entrée n'établit pas un gain robuste.

Décision : je retiens l'époque 3 et je la déploie. Un assistant qui a perdu ses connaissances médicales générales n'est pas un socle acceptable, et le mixte fait au moins aussi bien sur la macro-F1, l'ESI exact (75,2 % contre 73,1 %), le sur-triage (18,1 % contre 21,8 %) et la cohérence bilingue. J'accepte en connaissance de cause quatre entrées urgentes manquées en plus sur 100, toutes des ESI 2 classés 3 ; c'est l'erreur que le DPO ciblé cherche ensuite à corriger. Je ne prolonge plus la même recette sans nouvelle hypothèse.

8. Préparer un DPO utile au triage

Ce que le DPO cherche à apprendre

Le SFT apprend à reproduire une réponse cible. Le DPO compare deux réponses au même cas et augmente leur préférence relative par rapport à un modèle de référence figé. Une amélioration de la loss de préférence n'implique pas automatiquement une amélioration du triage : le premier pilote l'a montré.



Je prépare uniquement des paires issues du train triage. Le modèle produit d'abord 1 888 réponses, avec empreinte du checkpoint et des données. Je retiens une erreur ESI seulement si la sortie est complète et cohérente. Les sous-triages critiques, les autres sous-triages et les sur-triages sont comptés séparément. Je n'invente pas de réponse rejetée pour compléter un quota.

Je réserve 20 % des groupes de cas au contrôle interne DPO, en gardant FR et EN ensemble. Ce contrôle reste issu du train SFT ; il ne mesure pas une généralisation indépendante. La validation métier utilise les 500 exemples v3 restés hors entraînement. Les étiquettes restent silver et les écarts de style entre réponses peuvent influencer les préférences.

Paramètres et critère de réussite

Je réalise 32 mises à jour, taux 0,000002, beta 0,1, microbatch 1 et accumulation 16. Les scores métier sont calculés pour les pas 16 et 32, ainsi que pour la référence SFT dans le même environnement. Les fichiers de test ne figurent pas dans le paquet Colab.

Le signal favorable exige : rappel urgent en hausse, recul de macro-F1 limité à 1 point, hausse du sur-triage limitée à 2 points, recul QA limité à 3 points et format ESI sans recul. Ces tolérances sont des choix expérimentaux, pas des seuils cliniques ; elles n'autorisent aucune promotion automatique.

Résultats : run terminé sur G4 en 723 s. Les 1 500 prédictions et les six rapports concordent après recalcul local. Les poids source et DPO 16/32 sont conservés ; aucun test final n'a été relu.

Validation v3 du même run	SFT mixte	DPO 16	DPO 32
QCM / F1 QA	56,49 % / 0,4322	60,31 % / 0,4371	58,78 % / 0,4333
Macro-F1 priorité	0,7875	0,7457	0,7373
Rappel urgent	91/100	87/100	83/100
Sur-triage	18,49 %	14,71 %	12,18 %
Cohérence bilingue	78,15 %	74,79 %	71,43 %

Le minage retient 440 erreurs, séparées en 355 paires de train et 85 de contrôle interne. Le train contient 234 sur-triages (66 %), 72 sous-triages critiques et 49 autres sous-triages. Ce déséquilibre peut expliquer une baisse de l'urgence prédite ; ce n'est pas une preuve causale. Les deux checkpoints échouent aux critères de rappel et macro-F1. Je conserve la référence SFT mixte. Une nouvelle référence clinique indépendante reste nécessaire.

8.1 Sécurité, hallucinations et biais

Risque	Ce que j'observe	Parade dans le POC	Reste à faire
Sous-triage	9 urgences sur 100 classées ESI 3 en validation (7 cas)	Proposition toujours soumise au soignant ; rappel urgent suivi comme critère de sélection	Référence clinique, seuil d'acceptation fixé par les soignants
Hallucination dans la justification	Fièvre absente du cas citée en validation ; le 10 novembre, « hypotension » citée pour une tension saisie à 150/90, à l'identique sous Transformers et sous vLLM, avec un ESI 2 pourtant cohérent	Avertissement systématique ; le parseur ne contrôle que la structure	Vérifier que chaque élément cité figure dans la saisie
Réponse dangereuse hors triage	La QA plafonne à 57 % de QCM corrects	L'interface ne propose que le parcours de triage	Refus explicite des questions thérapeutiques
Biais d'annotation	Étiquettes produites par trois LLM, cas ambigus exclus	Consensus jusqu'à l'ESI exact, abstention autorisée	Double lecture humaine d'un échantillon
Biais de langue et de population	Écart FR/EN variable selon la partition ; vignettes d'examen, peu de pédiatrie ou de gériatrie réelles	Mesures par langue, paires FR/EN groupées	Cas réels représentatifs du service
Détournement de l'entrée	Balises de rôle et entrées trop longues	Refus 422 sans appel au modèle	Tests d'intrusion dédiés

Deux exemples résument le comportement. **Succès** : douleur thoracique constrictive avec sueurs, ESI 2 et priorité maximale dans les deux moteurs. **Échec** : entorse de cheville chez une patiente qui marche, classée ESI 3 avec biologie et imagerie anticipées, alors que la consigne ESI du projet conduit plutôt à une seule ressource, donc ESI 4 ; c'est un sur-triage probable, sans danger mais coûteux, que je n'ai pas fait relire par un soignant. Une sortie bien formée peut donc rester fausse : le format ne vaut pas validation.

9. Déployer un prototype traçable

Je sépare la collecte, le moteur d'inférence et l'audit. Le questionnaire demande les champs manquants et adapte une question à la présence lexicale de douleur. Il ne constitue pas un protocole clinique et ne délègue pas la conduite de l'entretien au LoRA. Le mode cas complet transmet directement la présentation saisie.

Stack et responsabilités

Couche	Outils	Rôle
Préparation	Python, pandas, PyArrow, JSONL	Exploration, conversion, contrôles et partitions
Modélisation	PyTorch, Transformers, PEFT, TRL	Socle Qwen3, LoRA SFT et DPO
Expérimentation	Colab, Drive, MLflow, SHA256	GPU temporaire, sauvegarde, paramètres et métriques
Service	FastAPI, Pydantic, vLLM, SQLite	Contrat HTTP, inférence et audit métier
Interface	Gradio, Transformers/PEFT, HF ZeroGPU	Démonstration bilingue gratuite
Livraison	GitHub Actions, Docker	Tests et publication du paquet autorisé

Deux voies techniques

Voie	Chaîne	Statut
Démonstration gratuite	Gradio → Transformers / PEFT → ZeroGPU	Fonctionnelle sur Hugging Face
API du POC	FastAPI → vLLM + LoRA → contrôle → SQLite	Vérifiée sur A100 puis en HTTPS public sur L4 le 10 novembre

Le sélecteur FR/EN traduit l'interface et réinitialise le cas. Les réglages exposent le mode guidé ou complet et les détails techniques ; les paramètres de génération restent fixés. Les deux voies lisent serving/modele.json : le SFT mixte époque 3 y est identifié par son SHA256 (62d4e508…), vérifié avant le démarrage de vLLM et avant chaque publication du Space.

L'API protège consultations et traces par une clé. Elle vérifie le modèle disponible, la longueur d'entrée et la cohérence ESI/priorité. Une sortie interrompue ou invalide conduit à une revue humaine sans priorité. Une panne du journal empêche la délivrance d'une proposition. Le contrat JSON simule une intégration SIH, sans connecteur hospitalier réel.

Je complète l'audit métier par un journal HTTP SQLite : début et fin de chaque requête, refus 401/422 inclus, appels vLLM, statuts, durées et type d'erreur. Un UUID généré côté API relie ces événements et figure dans X-Request-ID et la réponse. Les clés, corps de requête et paramètres d'URL restent exclus des logs techniques. L'audit métier conserve uniquement les cas fictifs de la démonstration. Une panne du journal bloque la délivrance d'une proposition ; les tests couvrent ce comportement.

Les traces SQLite sont en clair et ne sont pas immuables ; la clé partagée n'identifie pas individuellement les soignants. Le Space propose une trace JSON par session, téléchargeable, avec nettoyage temporaire du cache. Ce mécanisme ne remplace pas un audit hospitalier persistant.

9.1 Vérifications réalisées

Les tests locaux contrôlent données, métriques, formats, API et Space ; les tests du DPO ciblé ajoutent les contrôles de provenance, de séparation des cas et des garde-fous. La suite actuelle exécute 52 tests avec Pytest, avant toute étape de déploiement ; le premier run CI en avait exécuté 35 sur les données, métriques, API et Space, construit Docker et publié le paquet autorisé. Le test du conflit de port est ajouté à cette suite. Le run GitHub Actions 42074793162 est réussi : la révision attendue du Space est effectivement démarrée. Aucun corpus ni secret n'est envoyé dans ce paquet.

Avec le SFT successif, deux parcours fictifs FR/EN ont produit une inférence réelle : 5,115 s et 3,870 s côté serveur, appel ZeroGPU inclus. Il s'agit de deux observations, pas d'un test de charge. Après la bascule du 10 novembre, je vérifie le SFT mixte sur deux parcours fictifs en local (mêmes fonctions que le Space, GPU 24 Go) puis sur le Space redéployé, révision f32b2a65 : le parcours français aboutit à la même proposition ESI 2 dans les deux cas. Un essai ultérieur du mode direct a atteint le quota invité ; l'interface a présenté une indisponibilité sans priorité.

La vérification vLLM réelle sur A100 40 Go, faite le 5 novembre avec le SFT successif, mesure 6,935 s, 0,385 s et 0,514 s par requête HTTP, soit une médiane de 0,514 s. Le premier appel comprend potentiellement des initialisations. Les trois traces SQLite concordent avec les réponses ; les contrôles HTTP renvoient 401 sans clé, 422 pour une entrée invalide et une question pour un cas incomplet.

Le cas anglais produit « delayed emergency » au lieu du libellé attendu « delayed urgency ». L'API retourne une revue humaine sans priorité. Cette observation montre une fragilité du contrat linguistique ; je ne relâche pas le contrôle après le test. Trois requêtes ne prouvent ni la capacité en charge ni la pertinence clinique.

La démonstration permanente utilise Transformers. L'essai vLLM du 5 novembre est interne à une VM Colab temporaire ; la vérification publique du 10 novembre est décrite ci-dessous. Aucun endpoint vLLM permanent n'est maintenu. Le conflit initial sur le port 8080 est corrigé en réservant un port libre transmis à Uvicorn ; un test CPU reproduit ce conflit.

Endpoint vLLM public du 10 novembre

Le notebook 09 démarre vLLM 0.32.0 et l'API FastAPI sur une L4 de 23 Go louée sur Colab, avec le SFT mixte époque 3, puis les expose par un tunnel HTTPS protégé par clé. La vérification automatique passe par l'URL publique : trois cas fictifs séquentiels répondent en 7,74 s, 0,90 s et 1,23 s (médiane 1,23 s ; le premier appel inclut des initialisations), puis huit requêtes simultanées aboutissent toutes en 1,54 s. Depuis mon poste, hors Colab et sans clé, /healthz répond 200 et les routes protégées 401. Un parcours saisi dans la page de démo aboutit en 1,24 s à la même proposition ESI 2, mot pour mot, que Transformers en local et sur le Space, hallucination comprise : elle vient donc du modèle et non du moteur.

Le journal d'audit contient les 16 interactions de la session : 8 propositions, 4 demandes de complément et 4 revues humaines. Ces quatre revues concernent le même cas anglais, pour lequel le modèle écrit « delayed emergency » au lieu du libellé attendu : le défaut déjà vu le 5 novembre persiste avec le mixte et je ne relâche pas le contrôle. Le journal HTTP relie 30 requêtes entrantes à leurs appels vLLM, refus 401 inclus, sans clé ni corps. J'ai arrêté le service après une vingtaine de minutes ; l'URL n'existe plus. Ce n'est ni un hébergement permanent ni un test de charge.

9.2 Coût et performance de l'infrastructure

Entraînement

Calcul sur Colab, évaluations comprises	Durée observée
SFT successif, phase triage (357 pas)	3 min 42 s
DPO généraliste, 32 pas	1 min 40 s
SFT mixte, 3 époques et 3 validations (G4)	16 min 25 s
Poursuite, 7 époques et 7 validations	46 min 49 s
DPO ciblé, minage, 32 pas et 3 validations (G4)	12 min 03 s

Ces cinq calculs totalisent 1 h 21 de GPU. En ajoutant la phase QA, le test final, les essais interrompus et les installations, je reste à quelques heures de GPU. À titre d'ordre de grandeur, une A100 40 Go consomme environ 5,4 unités Colab par heure, soit près de 0,55 $ de l'heure à 10 $ les 100 unités (relevé tiers de mars 2026, variable) : l'ensemble des entraînements coûte quelques dollars. Une époque du SFT mixte prend environ 4 minutes ; la validation des 500 exemples, 77 secondes. Le premier essai mixte sur A100 40 Go a dépassé la VRAM au pas 62 avec un contexte de 5 120 tokens : la longueur des réponses QA, pas la taille du modèle, dimensionne la mémoire d'entraînement.

Inférence

Moteur	Matériel	Mesure	Lecture
vLLM + FastAPI, SFT successif	A100 40 Go, appels internes	0,385 s et 0,514 s par requête après un premier appel à 6,9 s	51 et 73 tokens générés, soit de l'ordre de 130 à 140 tokens/s en séquentiel
vLLM + FastAPI, SFT mixte	L4 23 Go, URL HTTPS publique	0,90 s et 1,23 s par requête après un premier appel à 7,7 s	50 et 70 tokens générés, soit environ 60 tokens/s en séquentiel
Même endpoint, 8 requêtes simultanées	L4 23 Go	8 réponses en 1,54 s : 5,2 requêtes/s et 329 tokens/s	Latence par requête inchangée (0,9 à 1,4 s) : le traitement par lots multiplie le débit par cinq environ
Transformers + PEFT	ZeroGPU	3,9 s et 5,1 s par proposition	Inclut l'attribution du GPU à chaque appel

C'est le bénéfice attendu de vLLM : huit patients simultanés attendent à peine plus qu'un seul. Cette mesure porte sur huit requêtes et un seul essai ; elle ne dit rien d'une charge soutenue ni de la saturation. L'écart avec ZeroGPU tient au moteur et à l'attribution du GPU, que je n'ai pas mesurés séparément. Sur Colab, une L4 consomme environ 1,7 unité par heure, soit près de 0,17 $ (relevé tiers) : la session de vérification a coûté quelques centimes.

Déploiement

Option	Coût indicatif	Usage
Space ZeroGPU	0 € ; quotas et file d'attente	Démonstration
Colab + tunnel, à la demande	Environ 0,17 $ par heure de L4, 0,55 $ par heure d'A100	Soutenance, essais ponctuels
Endpoint dédié, GPU L4 24 Go	Environ 0,80 $ par heure (tarif public Hugging Face Inference Endpoints), soit près de 580 $ par mois en continu, ou 190 $ à 8 h par jour	Pilote

Un modèle de 1,7 milliard de paramètres en BF16 occupe moins de 4 Go de VRAM : un GPU de 24 Go suffit et laisse de la marge pour le traitement par lots. Un socle 32B changerait d'échelle : plusieurs GPU ou une quantification, et un coût horaire plusieurs fois supérieur. Ces montants sont des ordres de grandeur publics, pas des devis ; un hébergement certifié HDS, obligatoire avec des données de patients, coûtera davantage.

10. Conservation, limites et recommandations

Conservation des expériences

Je récupère 363 fichiers Colab, soit environ 2,82 Go, avec contrôle des tailles et des SHA256. La copie MLflow locale contient 28 runs, dont 26 terminés et 2 échoués. Les URI d'artefacts sont adaptées dans une copie de la base ; les originaux restent conservés.

Les résultats du DPO généraliste sont disponibles, mais les derniers poids, le checkpoint 32 et la base après le DPO ne figuraient pas dans la sauvegarde distante accessible. La cause reste à établir. Je ne confonds pas un run observé avec la conservation complète de ses fichiers. La fermeture Colab attend maintenant la synchronisation Drive avant la libération de la VM.

Les quatre runs du test final sont récupérés dans une seconde copie MLflow locale. Les archives du test et de vLLM, les journaux et les traces SQLite sont conservés avec leurs empreintes. La vérification Drive confirme l'absence des poids finaux du DPO généraliste ; je conserve les résultats disponibles sans remplacer le checkpoint sélectionné par un autre.

Le run DPO ciblé du 6 novembre est récupéré séparément : 95 fichiers vérifiés par empreinte, poids source et DPO aux pas 16/32, prédictions, paires et base MLflow intègre. Les états optimiseur, calendrier, RNG et les copies du modèle de référence sont également récupérés et vérifiés.

Montrer le suivi MLflow

Depuis la racine du dépôt, .venv/bin/python scripts/ouvrir_mlflow.py démarre l'interface locale et ouvre le navigateur. L'option --experience sélectionne mixte, poursuite, historique final ou dpo-cible. Le serveur écoute uniquement sur 127.0.0.1 ; Ctrl+C l'arrête. Une copie de consultation protège les bases archivées.

Les checkpoints de la poursuite SFT restent sur Drive et ne sont pas tous rapatriés ; les fichiers du manifeste DPO ciblé sont, eux, entièrement récupérés. Les courbes et paramètres présents dans SQLite sont consultables ; cela ne prouve pas que les fichiers associés sont tous disponibles. L'ouverture du serveur a révélé une incompatibilité avec SQLAlchemy 1.4 : le projet exige maintenant SQLAlchemy 2.x.

La revue des dépendances a conduit à actualiser Flask, Gunicorn et Waitress dans le verrou de travail. Les alertes liées à Datasets/fsspec et cryptography restent contraintes par les versions de Datasets et Presidio ; je ne déclare pas cet environnement prêt pour la production. MLflow reste local et les paquets du Space et de FastAPI sont séparés.

Conditions et écarts restants

Niveau	Condition
Preuves techniques	Test final et vLLM vérifiés ; récupération finale DPO généraliste incomplète
Conformité au livrable cloud vLLM	Endpoint public vérifié le 10 novembre sur GPU Colab ; il est temporaire et se relance par le notebook 09, sans hébergement permanent
Données	Clarification des licences et revue humaine de l'anonymisation
Pertinence clinique	Évaluation indépendante par des professionnels, notamment des erreurs graves
Usage réel	Accès par rôle, journal sécurisé, supervision, protection des données et gouvernance

10.1 Priorités après ces expériences

1. Préserver les preuves finales et expliquer les écarts : sauvegarde DPO généraliste incomplète et hébergement public Transformers. Ne pas relancer un entraînement pour masquer un résultat défavorable du test.

2. Constituer une référence clinique indépendante : cas ambigus, urgences rares, informations absentes, négations et situations hors distribution.

3. Conserver le SFT mixte époque 3 en démonstration et ne pas retenir le DPO ciblé défavorable. Une éventuelle suite comparerait des préférences mieux équilibrées et relues, sans prolonger arbitrairement les époques.

4. Faire relire les préférences et les erreurs importantes par des professionnels avant toute conclusion clinique.

5. Tester le système complet : questionnaire, explications, abstention, charge, latence et comportement lors des pannes.

6. Définir un pilote encadré avec validation humaine, seuils d'acceptation, surveillance et retour arrière avant toute extension.

Un modèle de 32B+ reste une piste de projection. Il ne corrige pas automatiquement les biais d'annotation, les fuites ou les défauts de protocole. Je comparerais ses bénéfices à ceux d'une meilleure référence et d'un apprentissage mieux ciblé.

Conditions de passage au pilote (go / no-go)

Condition	Critère proposé	État
Référence clinique	Jeu indépendant relu par des soignants, urgences rares incluses	À constituer
Sécurité	Aucun ESI 1 manqué ; taux de sous-triage ESI 2 fixé et validé par les urgentistes	Non atteint : 9 sur 100 en validation silver
Fidélité	Justifications sans élément absent de la saisie, sur un échantillon relu	Non atteint : deux cas relevés
Service	Endpoint vLLM dédié, latence et débit mesurés sous charge, reprise après panne	Partiel : latence et débit sur 8 requêtes, endpoint temporaire
Données	Hébergement HDS, journal chiffré, accès nominatif, analyse d'impact	À faire
Exploitation	Supervision, alertes sur le taux de revue humaine et la dérive, retour arrière	À faire

Tant qu'une ligne reste ouverte, ma recommandation est de ne pas exposer le système à des patients : le POC reste un démonstrateur.

Annexe. Retrouver les preuves et reproduire la démarche

Emplacement	Fonction
data_ingestion/01_preparation_donnees.ipynb	Exploration, qualification, consensus et corpus
evaluation/02_evaluation_initiale.ipynb	Socle, métriques et protocole
evaluation/03_execution_colab.ipynb	Corrections, SFT, DPO et choix sur validation
serving/04_poc_et_deploiement.ipynb	Questionnaire, API, audit et déploiement
evaluation/05_finalisation_colab.ipynb	Test figé, vérification vLLM et sauvegarde
evaluation/06_verification_vllm_colab.ipynb	Reprise vLLM sur port libre, sans réévaluer le modèle
evaluation/07_sft_mixte_colab.ipynb	SFT mixte, poursuite et analyse des erreurs
evaluation/08_dpo_triage_cible_colab.ipynb	DPO ciblé terminé, comparaison et non-sélection
reports/bilan_final_20261105.json	Synthèse des mesures et empreintes des preuves finales
data/processed/partitions_v3	Partitions nettoyées et manifeste d'exclusions
data/evaluation/colab_sft_dpo_v3	Prédictions et bilan de validation
data/colab_archive/original	Fichiers distants conservés sans réécriture
data/colab_archive/local	Copie MLflow avec URI d'artefacts locales

Je repars de la révision du socle et du hash de l'adaptateur inscrits dans serving/modele.json. Je ne substitue pas un checkpoint voisin à des poids manquants. La seed est conservée, mais ne garantit pas une identité numérique entre moteurs ou versions des bibliothèques.

Le script evaluation/restaurer_colab.py vérifie les transferts et adapte les chemins MLflow dans une copie. Les paramètres historiques restent inchangés. Le script evaluation/test_final.py vérifie les empreintes avant l'inférence, conserve les réponses JSONL et journalise les métriques et artefacts.

L'entraînement est séparé du déploiement : le workflow de publication vérifie les poids retenus, sans réentraîner le modèle. La récupération d'un paquet identique peut aboutir au même commit Hugging Face ; je vérifie la révision réellement servie.

Sources et provenance

Les notebooks 01 à 09, les manifestes de partitions, bilan_v3.json et les protocoles d'entraînement constituent les preuves locales. Les annotations proviennent de LLM ; la préparation du code et de la documentation est assistée par des outils d'IA. Je n'attribue pas à une revue clinique les contrôles automatiques effectués.

Référence DPO : https://huggingface.co/docs/trl/dpo_trainer. Références de service : docs.vllm.ai/en/v0.32.0/getting_started/installation/gpu/ ; huggingface.co/docs/hub/spaces-zerogpu ; huggingface.co/docs/hub/spaces-github-actions. Tarifs : huggingface.co/docs/inference-endpoints/pricing ; mccormickml.com/2024/04/23/colab-gpus-features-and-pricing/. Démonstration : huggingface.co/spaces/rayakevin/ocr14-triage-fr-en. CI : github.com/rayakevin/OCR_PROJET_14/actions/runs/42074793162.

Accès aux ressources du projet

Je versionne le corpus et les adaptateurs dans deux dépôts Hugging Face privés : https://huggingface.co/datasets/rayakevin/ocr14-medical-bilingue-v3 et https://huggingface.co/rayakevin/ocr14-triage-lora-experiences. Les révisions sont inscrites dans reports/publication_livrables_hf.json. Le chargement distant des quatre configurations du dataset est vérifié. Les métadonnées absentes sont uniformisées à null pour HF Datasets ; les textes et partitions restent identiques.

Le dépôt dataset expose QA, triage, préférences générales et préférences ciblées. La racine du dépôt modèle contient le SFT mixte époque 3, déployé ; les sous-dossiers gardent le successif v3, évalué sur le test final, et les DPO ciblés 16/32, explicitement non retenus. Les fichiers empreintes.json permettent de contrôler le contenu. Le dépôt privé ne résout pas les restrictions de licence ; l'accès distant nécessite une autorisation, et les fichiers sont inclus dans la remise.

La présentation graphique explique les données, les entraînements, les résultats et l'architecture. Les graphiques publics sont agrégés, sans cas individuel. La vue interactive contient des exports MLflow ; le serveur MLflow complet reste local et nécessite les archives d'expériences, qui ne sont pas incluses dans Git.

Le notebook 09 ouvre une voie vLLM publique temporaire : Colab, API authentifiée, tunnel HTTPS, trois requêtes fictives de contrôle, mesure de débit, sauvegarde puis arrêt. Son exécution du 10 novembre est conservée avec ses journaux. Une URL de production stable nécessiterait un hébergement GPU dédié, une supervision et un budget ; je ne présente pas un tunnel de démonstration comme un service permanent.

Dans la remise du 12 novembre 2026, je fournis les datasets, les adaptateurs SFT et DPO ciblés, le rapport PDF, la présentation HTML et le nécessaire au lancement de l'endpoint vLLM temporaire. Le dépôt GitHub public donne accès au code, aux notebooks et au pipeline CI/CD. Les données et poids sont inclus car leurs dépôts HF sont privés. Les secrets et notes personnelles sont exclus. La vérification publique du 10 novembre est réalisée ; une démonstration en direct nécessite de relancer le notebook 09.